D

Dumbhanengue ERP

07 — Sécurité

Ce document couvre les décisions de sécurité de l'app : **stockage sécurisé** des secrets, limites de responsabilité entre l'app et Supabase, et bonnes pratiques par plateforme.

Version 1.1.1+3 · 16 août 2026

Dumbhanengue ERP

Table des matières

Stockage sécurisé (`SecureStorageService`) 3

- Architecture 3
- Pourquoi ne pas utiliser `shared_preferences` pour les données sensibles ? 3
- Pourquoi cela protège le compte du commerçant 3

La session Supabase vit maintenant dans le backend sécurisé (S1, Aug 6) 3

RLS (Row Level Security) sur Supabase 4

Particularités par plateforme (exactitude technique) 5

Autres pratiques du projet 5

Documents connexes 5

Stockage sécurisé (SecureStorageService)

lib/core/security/secure_storage_service.dart encapsule la persistance chiffrée des jetons d'accès, des jetons de rafraîchissement et des clés API.

Architecture

```
SecureStorageService
  SecureStorageBackend (interface)        ← abstraite pour les tests
    FlutterSecureStorageBackend           ← réel : flutter_secure_storage
    (backend en mémoire dans les tests)
```

- Clés centralisées dans SecureStorageKeys : accessToken, refreshToken, apiKey — évite les fautes de frappe et permet d'auditer exactement ce qui est traité comme sensible.
- API : write, read, delete, contains.
- Enregistré dans GetIt (ServiceLocator) — point d'accès unique pour les couches données/présentation.

Pourquoi ne pas utiliser shared_preferences pour les données sensibles ?

Aspect	shared_preferences	flutter_secure_storage
Emplacement physique	Fichier de préférences en texte clair	Keychain (iOS/macOS) et Android Keystore
Chiffrement	Aucun — valeur lisible sur disque	Chiffré ; sur Android, la clé (AES-GCM) est générée et conservée dans le Keystore, liée au matériel
Accès	Tout processus ayant accès en lecture au fichier	Protégé par l'OS ; sur iOS peut exiger la biométrie (Face ID/Touch ID)
Sauvegarde/extraction	Valeurs en texte clair lisibles depuis les sauvegardes, root/JB ou extraction APK	Chiffré de bout en bout
Cas d'usage idéal	Préférences UI (thème, langue, filtres)	Jetons de session, jetons de rafraîchissement, clés API, secrets

Pourquoi cela protège le compte du commerçant

1. Le jeton est le mot de passe. Avec le jeton de session (ou de rafraîchissement) en main, un attaquant accède au compte sans avoir besoin du mot de passe.
2. Chiffrement lié à l'appareil : la clé maîtresse vit dans l'Android Keystore (adossée au matériel, TEE) / Keychain avec enclave sécurisée — copier le fichier ne suffit pas sans la clé matérielle.
3. Coût d'attaque élevé : compromettre l'OS/le matériel n'est pas viable à l'échelle (la même confiance qu'utilisent les banques).
4. Accès conditionné à l'utilisateur (iOS) : le Keychain peut exiger la biométrie.

La session Supabase vit maintenant dans le backend sécurisé (S1, Aug 6)

La session Supabase (jetons d'accès + rafraîchissement) est persistée par supabase_flutter via un LocalStorage que nous fournissons — `SecureSupabaseLocalStorage` — qui délègue à `flutter_secure_storage` (Keychain/Keystore) sur les plateformes natives. Le vérificateur de code PKCE utilise le même backend (SecureGotrueAsyncStorage). Le défaut du SDK (shared_preferences, XML en texte clair) n'est pas utilisé sur natif ; sur web, le défaut du SDK (window.localStorage) est conservé car flutter_secure_storage 10.x n'a pas de support web.

SecureStorageService reste l'abstraction unique sur le backend sécurisé — il soutient le câblage d'authentification et les secrets de l'app :

- clés API des services externes (ex. : passerelle de paiement) ;
- secrets de configuration ;
- toute information d'identification que les couches données/présentation doivent conserver.

RLS (Row Level Security) sur Supabase

- La clé anon est publique par conception — embarquée dans le client. La vraie sécurité vient de la base :

```
-- règle d'isolation par tenant ET par magasin
user_id = auth.uid() OR public.is_my_store(store_id)
```

- `is_my_store(store_id)` : le propriétaire du tenant est valide pour tous les magasins du tenant ; un employé lié est valide pour son magasin (store_employees). Des triggers anti-falsification imposent user_id = auth.uid() et un store_id valide sur chaque insert.
- `is_manager_of_store(store_id)` (Aug 6) : matrice de niveau d'accès — le propriétaire du tenant ou un employé lié dont le store_employees.role = 'manager' a le pouvoir *managerial* sur le magasin. Utilisée par la garde de la RPC process_purchase et par les politiques RLS purchases/ purchase_items : un employé ordinaire peut opérer ventes/stock/crédit mais ne peut ni enregistrer ni lire les achats — imposé côté serveur, même si un client modifié appelle PostgREST directement.
- Les fonctions helper RLS (is_my_store, is_tenant_owner, is_manager_of_store) sont SECURITY DEFINER avec search_path épinglé à public (Aug 6) : elles sont appelées *dans* les politiques et interrogent des tables ayant leurs propres politiques (stores/tenants/store_employees), ce qui formait auparavant un cycle inter-tables pouvant faire exploser la pile (54001 max_stack_depth) sur les scans de table complète. Comme definers, elles contournent le RLS en interne, cassant le cycle par construction — même pattern que is_super_admin et les RPC d'argent. Elles ne renvoient qu'un booléen sur un id explicite, donc aucune donnée n'est exposée.

- Les RPC d'argent (process_sale, register_debt, settle_debt, process_purchase) sont SECURITY DEFINER et revalident l'accès en interne, puisque le definer contourne le RLS — avec search_path épinglé à public. process_sale/register_debt/settle_debt exigent is_my_store (tout employé lié) ; process_purchase exige is_manager_of_store (propriétaire ou gérant).
- Même si un bug de l'UI fuit des données, la base les filtre.
- Ne mettez jamais la clé *service role* dans le client (elle contourne le RLS).

Particularités par plateforme (exactitude technique)

Plateforme	Mécanisme	Niveau de protection
Android/iOS	Android Keystore (AES-GCM, clé dans le TEE) / Keychain avec enclave	Adossé au matériel — là où les données critiques devraient vivre
Windows	DPAPI (lié au compte utilisateur de l'OS)	Raisonnable, pas d'enclave dédiée
Linux	libsecret	Raisonnable, pas d'enclave dédiée
Web	Clé de chiffrement conservée dans localStorage	Le plus faible — pas adossé au matériel

> Pour les secrets critiques, privilégiez le mobile ; sur le web, restreignez ce qui est stocké ou combinez avec des mesures côté serveur.

Autres pratiques du projet

- Erreurs d'authentification génériques dans l'UI : debugPrint avec détails pour le diagnostic ; SnackBar conviviale pour l'utilisateur (n'expose pas les internals du provider).
- Règles métier dans le domaine, gardes dans la base : les valeurs non négatives sont validées dans les cas d'usage et par les contraintes CHECK en SQLite (défense en profondeur).
- SQL paramétré dans tous les datasources SQLite (instructions préparées — jamais de concaténation de chaînes), empêchant l'injection SQL.

Documents connexes

- Authentification — flux de session et où vit chaque jeton.
- Couche de données — schéma et gardes de la base.
- Architecture — où la sécurité s'insère dans la structure.